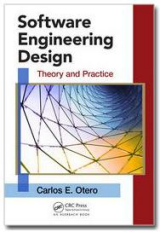
CHAPTER 1: INTRODUCTION TO SOFTWARE ENGINEERING DESIGN

SESSION III: SOFTWARE DESIGN FUNDAMENTALS

Software Engineering Design: Theory and Practice
by Carlos E. Otero



Slides copyright © 2012 by Carlos E. Otero

For non-profit educational use only

May be reproduced only for student use when used in conjunction with *Software Engineering Design: Theory and Practice.* Any other reproduction or use is prohibited without the express written permission of the author.

All copyright information must appear if these slides are posted on a website for student use.

SESSION'S AGENDA

- Roles of software designers
  - ✓ Systems Engineer
  - ✓ Software architect
  - ✓ Component designer
  - ✓ User interface designer

- Software design fundamentals
  - ✓ Design principles
    - Modularization
    - Abstraction
    - Encapsulation
    - Cohesion and coupling
    - Separation of interface and implementation
    - Sufficiency and completeness
  - ✓ Design strategies
    - Object-oriented vs. structured design
  - ✓ Practical design considerations

ROLES OF SOFTWARE DESIGNERS

- Systems Engineer
  - ✓ Designs systems using a holistic approach, which includes designing how software, hardware, people, etc. collaborate to achieve the system's goal.
- Software Architect
  - ✓ Design software systems using (for the most part) a black-box modeling approach; concern is placed on the external properties of software components that determine the system's quality and support the further design of functional requirements.
- Component Designer
  - ✓ Focuses on designing the internal structure and behavior of software components identified during the architecture phase; typically, these designers have strong programming skills, since they implement their designs in code.
- User Interface Designer
  - ✓ Design the software's user interface; skilled in determining ways that increase the usability of the system

SOFTWARE DESIGN FUNDAMENTALS

- Within the design process, many principles, strategies, and practical considerations exist to help designers execute the design process in effective and consistent manner.

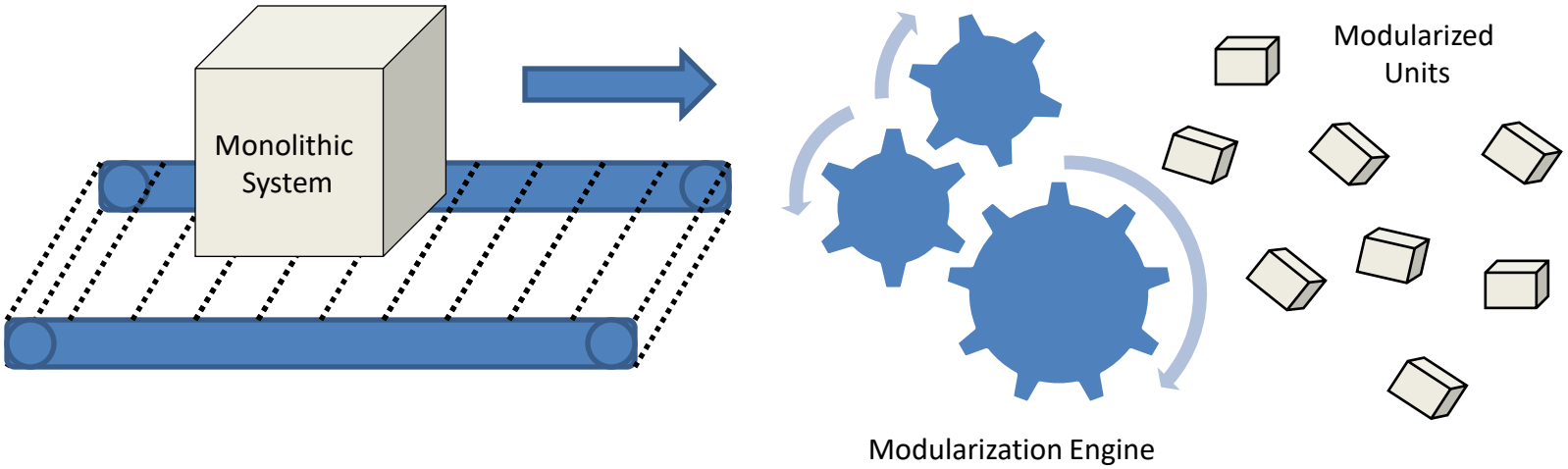
- General design principles
  - ✓ Refer to knowledge matter that has been found effective throughout the years in multiple projects on different domains.
  - ✓ Serve as fundamental drivers for decision-making during the software design process.
    - They are used as basis for reasoning and serve as justification for design decisions.
  - ✓ Provide designers a foundation from which other design methods can be applied.
  - ✓ Are not specific to particular design strategies (e.g., object-oriented) or process.
    - The are fundamental to every design effort.
    - Can be employed during architecture, detailed design, construction design, etc.

SOFTWARE DESIGN FUNDAMENTALS

- Design Principle #1: *Modularization*
  - ✓ It is the principle that drives the continues decomposition of the software system until fine-grained components are created.
  - ✓ One of the most important design principle, since it allows software systems to be manageable at all phased of the development life-cycle.
  - ✓ When you modularize a design, you are also modularizing requirements, programming, test cases, etc.
  - ✓ Plays a key role during all design activities; when applied effectively, it provides a roadmap for software development starting from coarse-grained components that are further modularized into fine-grained components directly related to code.
  - ✓ Leads to designs that are easy to understand, resulting in systems that are easier to develop and maintain.

Software Design Fundamentals - Modularization

- Conceptual view of modularization:



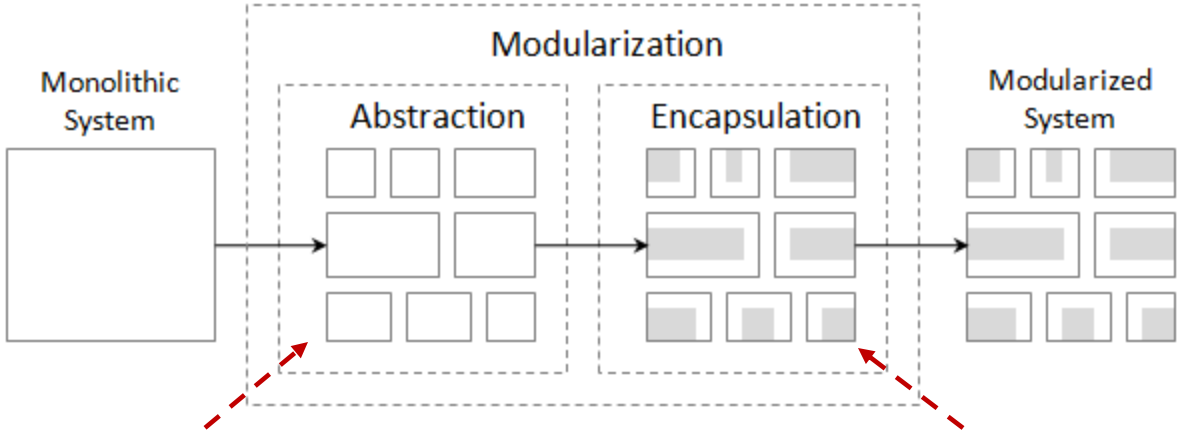
- Modularization is the process of continuous decomposition of the software system until fine-grained components are created. But how do we justify the "modularization engine"?
  - ✓ It turns out that two other principles can effectively guide designers during this process
    - Abstraction
    - Encapsulation

SOFTWARE DESIGN FUNDAMENTALS

- Design Principle #2: *Abstraction*
  - ✓ Abstraction is the principle that focuses on essential characteristics of entities—in their active context—while deferring unnecessary details.
  - ✓ While the principle of modularization specifies what needs to be done, the principle of abstraction provides guidance as to how it should be done. Modularizing systems in ad-hoc manner leads to designs that are incoherent, hard to understand, and hard to maintain.
  - ✓ Abstraction can be employed to extract essential characteristics of:
    - Data
    - Procedures or behavior
- *Procedural abstraction*
  - ✓ Specific type of abstraction that simplifies reasoning about behavioral operations containing a sequence of steps.
  - ✓ We use this all the time, e.g., consider the statement "*Computer 1* ***SENDS*** *a message to server computer 2*"
    - Image if we had to say, e.g., "*Computer 1 retrieves the server's information, opens a TCP/IP connection, sends the message, waits for response, and closes the connection.*" Luckily, the procedural abstraction *SEND* helps simplify the operations so that we can reason about this operations more efficiently.
- *Data abstraction*
  - ✓ Specific type of abstraction that simplifies reasoning about structural composition of data objects.
    - In the previous example, *MESSAGE* is an example of the data abstraction; the details of a *MESSAGE* can be deferred to later stages of the design phase.

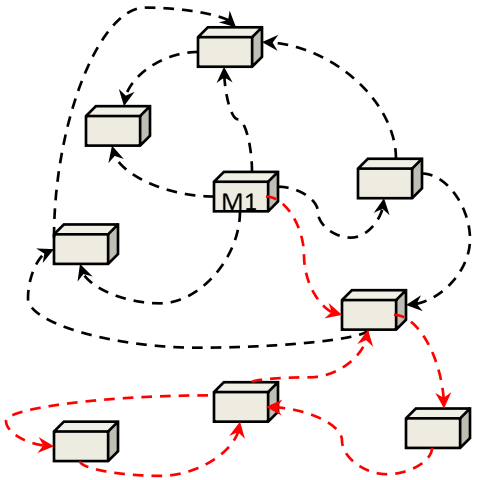
SOFTWARE DESIGN FUNDAMENTALS

- Design Principle #3: *Encapsulation*
  - ✓ Principle that deals with providing access to the services of *abstracted entities* by exposing only the information that is essential to carry out such services while hiding details of how the services are carried out.
  - ✓ When applied to data, encapsulation provides access only to the necessary data of abstracted entities, no more, no less.
  - ✓ Encapsulation and abstraction go hand in hand.
    - When we do abstraction, we hide details…
    - When we do encapsulation, we revise our abstractions to enforce that abstracted entities only expose essential information, no more, no less.
    - Encapsulation forces us to create good abstractions!
- The principles of modularization, abstraction, and encapsulation can be summarized below.

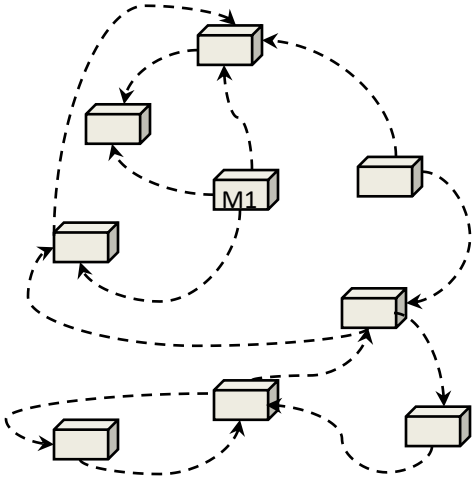


SOFTWARE DESIGN FUNDAMENTALS

- Design Principle #4: Coupling
  - ✓ Refers to the manner and degree of interdependence between software modules.
  - ✓ Measurement of dependency between units. The higher the coupling, the higher the dependency and vice versa.



Module 1 (M1) with higher coupling



Module 1 (M1) with lower coupling

SOFTWARE DESIGN FUNDAMENTALS

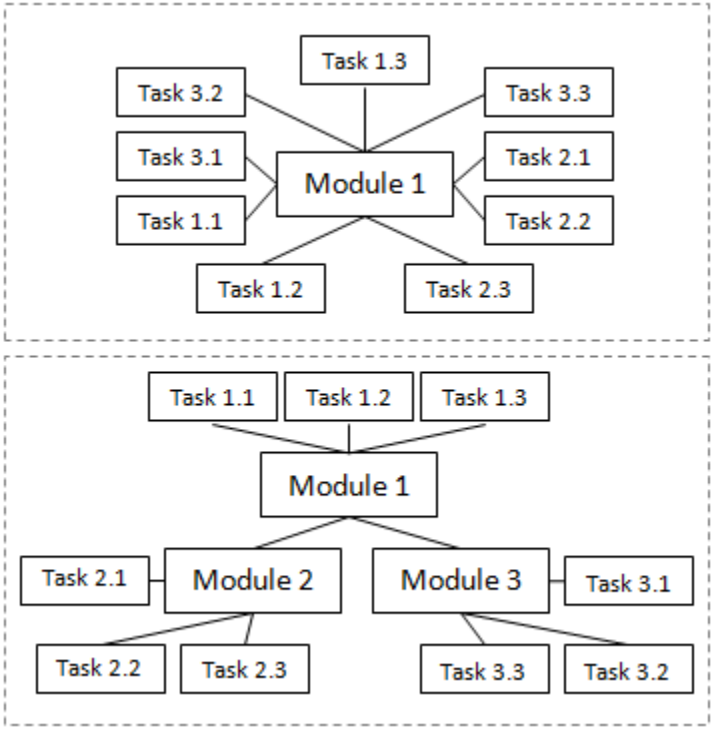
- Important types of coupling include:
  - ✓ Content coupling
    - The most severe type, since it refers to modules that modify and rely on internal information of other modules.
  - ✓ Common coupling
    - Refers to dependencies based on common access areas, e.g., global variables.
    - When this occurs, changes to the global area causes changes in all dependent modules.
    - Lesser severity than content coupling.
  - ✓ Data coupling
    - Dependency through data passed between modules, e.g., through function parameters.
    - Does not depend on other modules' internals or globally accessible data, therefore, design units are shielded from changes in other places.
- In all cases, a high degree of coupling gives rise to negative side effects.
  - ✓ Quality, in terms of reusability and maintainability, decrease.
  - ✓ When coupling increase, so does complexity of managing and maintaining design units.

SOFTWARE DESIGN FUNDAMENTALS

- Design Principle #5: *Cohesion*
  - ✓ The manner and degree to which the tasks performed by a single software module are related to one another.
  - ✓ Measures how well design units are put together for achieving a particular tasks.

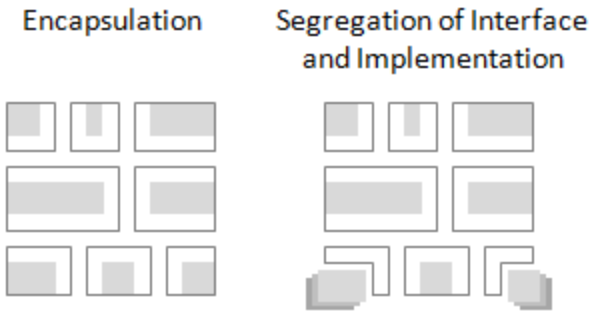
- Cohesion can be classified as:
  - ✓ Functional cohesion
  - ✓ Procedural (or sequential) cohesion
  - ✓ Temporal cohesion
  - ✓ Communication cohesion

- High cohesion good, low cohesion bad…



SOFTWARE DESIGN FUNDAMENTALS

- Design Principle #6: *Separation of Interface and Implementation*
  - ✓ Deals with creating modules in such way that a stable interface is identified and separated from its implementation.
  - ✓ Not the same thing as encapsulation!
  - ✓ While encapsulation dictates hiding the details of implementation, this principle dictates their separation, so that different implementations of the same interface can be swapped to provide modified or new behavior.



SOFTWARE DESIGN FUNDAMENTALS

- Design Principle #7: *Sufficiency*
  - ✓ Deals with capturing enough characteristics of the abstraction to permit meaningful interaction
  - ✓ Must provide a full set of operations to allow a client proper interaction with the abstraction.
  - ✓ Implies minimal interface

- Design Principle #8: *Completeness*
  - ✓ Deals with interface capturing all the essential characteristics of the abstraction.
  - ✓ Implies an interface general enough for any prospective client
  - ✓ Completeness is subjective, and carried too far can have unwanted results.

SOFTWARE DESIGN STRATEGIES

- Object-oriented design strategy
  - ✓ Design strategy in which a system or component is expressed in terms of objects and connections between those objects.
  - ✓ Focuses on object decomposition.
    - Objects have state
    - Objects have well-defined behavior
    - Objects have unique identity
  - ✓ Supports inheritance and polymorphism

- Structured (or Functional) design strategy
  - ✓ Design strategy in which a system or component is decomposed into single-purpose, independent modules, using an iterative top-down approach.
  - ✓ Focuses on
    - Functions that the system needs to provide,
    - the decomposition of these functions, and
    - The creation of modules that incorporate these functions.
  - ✓ Largely inappropriate for use with object-oriented programming languages.

PRACTICAL SOFTWARE DESIGN CONSIDERATIONS

- Design for minimizing complexity
  - ✓ Design is about minimizing complexity.
  - ✓ Every decision that is made during design must take into account reducing complexity [1].
  - ✓ When faced with competing design option, always choose the one that minimizes complexity

- Design for change
  - ✓ Software will change, design with extension in mind.
  - ✓ A variety of techniques can be employed through the design phase to achieve this.

WHAT'S NEXT…

- On the next session, we will explore a popular design language, the Unified Modeling Language, specifically,
  - ✓ What is UML?
  - ✓ UML Fundamentals
  - ✓ Structural Modeling
  - ✓ Behavioral Modeling
  - ✓ …
- UML will be heavily used to explain design concepts throughout the course.

REFERENCES

- [1] McConnell, Steve. Code Complete, 2d ed. Redmond, WA: Microsoft Press, 2004.